Password Attacks

Unit 14 — Module 3: Exploitation (final unit)

Passwords are still the front door to most systems — and most break-ins. We'll learn how they're stored, how attackers attack them, and how to **defend** them.

Curriculum by AJ Hammond — PNPT, CRTO, OSCP, BSCP

Where we are

- Passwords are the **front door** to most systems.
- Most real-world break-ins start with a **stolen or guessed password**.
- This unit shows **how passwords are stored**, how attackers go after them **online** and **offline**, and the **defenses** that actually work.

> Every attack here is paired with its defense. We learn the attack to protect people.

Learning objectives

By the end of this unit you can:

- **Explain** password storage: plaintext (bad), **hashing**, **salting**.
- **Identify** common hash types (**MD5/SHA** weak, **bcrypt** strong) and why.
- **Distinguish online** attacks (live login) from **offline** (stolen hashes).
- **Compare** brute force vs. **dictionary/wordlist** attacks (e.g., **rockyou**).
- **Use Hydra** against a **lab** login service.
- **Use John the Ripper** (and/or **Hashcat**) to crack **sample** hashes.
- **Describe password spraying** and why it dodges lockouts.
- **Recommend** defenses: long unique passwords, **MFA**, **lockout**, strong hashing.

Vocabulary (1 of 2)

Term	Meaning
Plaintext password	A password stored as readable text — a disaster if leaked.
Hash	A one-way scramble; same input → same hash, can't reverse.
Hashing	Turning a password into a hash so the real password isn't stored.
Salt	Random per-password data so identical passwords get different hashes.
MD5 / SHA	Older, **fast** hashes — easier to crack for passwords.
bcrypt	A **slow, salted** password hash that resists cracking.

Vocabulary (2 of 2)

Term	Meaning
Online attack	Guessing against a **live** login service (website, SSH).
Offline attack	Cracking **captured hashes** on your own machine.
Brute force	Trying every possible combination — thorough but slow.
Dictionary / wordlist	Trying likely passwords — faster, usually more effective.
Wordlist	A file of candidate passwords (e.g., **rockyou.txt**).
Hydra	A tool for online attacks against login services.
John / Hashcat	Tools for offline hash cracking (Hashcat is GPU-accelerated).
Password spraying	One common password against **many** accounts.
MFA / Lockout	Second factor / block after too many failed logins.

⚖️ Read this before anything else

The ethics & legal line

The bright line

- Attack **lab accounts on lab systems** and **authorized TryHackMe rooms** — and nothing else.
- **Never** a real account, a classmate's account, a school login, or any service you don't own or have **written permission** to test.
- Even **one** unauthorized login attempt can be a crime under the **CFAA** and state law.

> Cracked credentials from this lab are **lab data only** and never leave the lab.

A privacy note on rockyou

- The **rockyou** wordlist is **real passwords from a real breach**.
- It's proof that what people choose really does get exposed.
- Treat it as **evidence of why people need better defenses**, not a toy.

> We study these attacks to **defend** people — to push for MFA, lockouts, strong hashing, and long unique passwords.

Discussion: the "it's their own account" trap

> A friend forgot their email password and asks you to "just crack it for them — it's their own account."

- Is that okay? Why not?
- Where is the **authorization boundary**?
- What's the right answer instead?

Day 1

How passwords are stored: hashing & salting

Warm-up

When a website stores your password, what *exactly* is in the database?
Should it be the password itself?

Plaintext is a disaster

- **Plaintext** = the actual password stored as readable text.
- If the database leaks, **every real password is instantly exposed**.
- Worse: people reuse passwords, so one leak unlocks other sites too.

> Storing passwords in plaintext is the worst-case starting point. Everything we do next is to avoid it.

Hashing: the one-way scramble

- A **hash** turns a password into a fixed scramble.
- **Same input → same hash** (deterministic).
- But you **can't reverse** a hash back to the password.
- To check a login, the site hashes what you typed and compares hashes.

> Because it's one-way, an attacker can only **guess** a word, hash it, and compare — they can't "undo" the hash.

Hashing in action

```
echo -n "password1" | md5sum     # always the same hash
echo -n "password1" | md5sum     # identical to above
echo -n "Password1" | md5sum     # ONE change -> totally different
```

- The first two outputs are **identical** (same input → same hash).
- The third is **completely different** — one character changes everything.
- That sensitivity is called the **avalanche effect**.

Salting: make identical passwords differ

- A **salt** = random data added to each password **before** hashing.
- Two users who both pick `password1` end up with **different** hashes.

Why it matters:

- Defeats **rainbow tables** (precomputed hash lists).
- Stops an attacker from spotting that two users share a password.

> Salt makes every stored hash unique, even for identical passwords.

Weak vs. strong hashes

Algorithm	Speed	Good for passwords?
MD5 / SHA	very fast	❌ fast = easy to crack
bcrypt	slow + salted	✅ designed to resist cracking

Fast hashing helps the **attacker** (millions of guesses/sec). **Slow, salted** hashing is the goal — every guess costs the attacker real time.

Your turn (journal)

1. Hash **two** sample words and record the outputs.
2. Write **one sentence** on why hashing beats plaintext.

Day 1 exit ticket

Why does **salting** make a stolen password database harder to crack?

Day 2

Online vs. offline; brute force vs. wordlists

Warm-up

Would you rather attack a **live login** or a **stolen hash file**? Why?

Online vs. offline attacks

	Online	Offline
Target	a **live** login service	a **captured** hash file
Noise	loud, can trigger lockouts	silent — no contact with target
Speed	slow	as fast as your hardware
Tool	**Hydra**	**John**, **Hashcat**

Online = knocking on the real door. Offline = you stole the lockbox and crack it at home.

Brute force vs. wordlists

- **Brute force** — try *every* combination. Thorough, but very slow.
- **Dictionary / wordlist** — try a list of **likely** passwords. Much faster.

```
wc -l /usr/share/wordlists/rockyou.txt   # ~14 million lines
head /usr/share/wordlists/rockyou.txt    # 123456, password, iloveyou ...
```

> Wordlists win because people reuse predictable passwords. `rockyou.txt` is real breached passwords — a list of what humans actually pick.

Why length beats everything

- Brute force time **explodes** with each extra character.
- A short password in rockyou falls in **seconds**.
- A long, non-predictable passphrase isn't in any wordlist — and brute force would take longer than a lifetime.

Password	Likely outcome
`password1`	in rockyou → instant
`correct-horse-battery-staple-42`	not in any list; far too long to brute force

Password spraying

- Brute force = **many** passwords against **one** account → triggers lockout fast.
- **Spraying** = **one** common password against **many** accounts.

> Each account only sees *one* failed attempt — so simple lockouts don't notice. That's why spraying is sneaky.

Defense preview: unique passwords + MFA + monitoring for "one password, many accounts."

Your turn (lab setup)

1. **Read the Safety & authorization reminder aloud** (from `lab.md`).
2. Confirm `rockyou.txt` exists and is decompressed:

```
ls -l /usr/share/wordlists/rockyou.txt
gunzip /usr/share/wordlists/rockyou.txt.gz    # only if still .gz
```

3. **Inspect** the Hydra command — but don't run it yet.

Day 2 exit ticket

Give **one advantage of an offline attack** and **one of an online attack**.

Day 3

Online attack with Hydra (lab login only)

Warm-up

What does an attacker need to **start guessing** a login?

Hydra basics

Hydra automates guessing against a **live** login service.

You give it:

- a **username** (`-l`),
- a **wordlist** (`-P`),
- the **service** and **target** (e.g., `ssh://<ip>`).

> Re-state every time: **lab account on a lab system only.**

Running Hydra (lab login ONLY)

```
# SSH
hydra -l <lab-user> -P /usr/share/wordlists/rockyou.txt ssh://<target-IP>

# FTP
hydra -l <lab-user> -P /usr/share/wordlists/rockyou.txt ftp://<target-IP>
```

Success line:

```
[22][ssh] host: <ip>  login: <user>  password: <found>
```

For a web form, the instructor gives you the exact `http-post-form` string.

What Hydra is doing (and what stops it)

- It sends login attempt after login attempt until one works.
- This is exactly what **account lockout / rate-limiting** is designed to stop.
- And **MFA** means a guessed password **still fails** without the second factor.

> A loud, repetitive attack like this is *meant* to be caught by defenses.

Guided practice — watch it find the password

- Paced class demo: run Hydra against the **lab** login service with a small wordlist.
- Watch it report the valid login.

Your turn (lab)

1. Run Hydra against the **authorized lab service** (or THM room).
2. Capture the found credential in your worksheet.
3. In your journal, name **two defenses** that would have stopped it and **how**.

Day 3 exit ticket

Name **two defenses** that directly defeat a Hydra-style online attack, and **how** each works.

Day 4

Offline cracking with John the Ripper / Hashcat

Warm-up

You stole a file of hashes. The target has no idea.
How do you turn hashes into passwords?

Offline cracking, step by step

1. Save the **captured (sample) hashes** to a file.
2. Identify the **hash type** (MD5? SHA-1? bcrypt?).
3. Feed the hashes + a **wordlist** to **John** (or Hashcat).
4. The tool hashes each guess and compares — **matches = cracked**.

> No contact with the target. Limited only by your hardware and the hash algorithm.

Cracking with John the Ripper

```
nano hashes.txt          # paste the sample hashes (one per line)

john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hashes.txt
john --show --format=raw-md5 hashes.txt          # see what cracked
```

- Match `--format` to the hash type (`raw-md5`, `raw-sha1`, `bcrypt`).
- `--show` prints the cracked plaintext next to each solved hash.

> Sample hashes are made from harmless words — **never** real users' passwords.

Optional: Hashcat (GPU speed)

```
hashcat -m 0 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt
```

- `-m 0` = MD5 · `-a 0` = straight wordlist attack.
- Hashcat uses the **GPU**, so it guesses far faster than John on a CPU.

Watch the algorithm matter

- **MD5 / SHA** samples crack **almost instantly** (millions–billions of guesses/sec).
- The **bcrypt** sample is **slow** or won't crack in class time.
- Why? bcrypt is **slow by design and salted** — each guess costs far more time.

> This is the bridge to the defense: **bcrypt makes offline cracking impractical.**

Your turn (lab)

1. Crack the provided **sample hashes** with John.
2. Record which **cracked**, which **didn't**, and the plaintext for the ones that did.
3. Note **why** the bcrypt sample resisted.

Day 4 exit ticket

Why did the **bcrypt** sample resist cracking when the **MD5** one fell fast?

Day 5

Defenses + document it (ends Module 3)

Warm-up

After this week, what password advice would you actually give your **family**?

Map each attack to its defense

Attack	Defense	Why it works
Hydra online guessing	**lockout / rate-limit** + **MFA**	blocks/slows guesses; stolen pw isn't enough
rockyou wordlist	**long unique passphrases**	not in the list, too long to brute force
John offline (MD5/SHA)	**bcrypt** (slow, salted)	each guess costs huge time
password spraying	**unique passwords + MFA + monitoring**	one pw can't open many doors

No journal is complete without the **defense** column.

The everyday defenses

- **Long, unique passphrases** — beat wordlists and brute force.
- **MFA** — a stolen password alone isn't enough.
- **Lockout / rate-limiting** — kills online guessing, slows spraying.
- **Strong salted hashing (bcrypt)** — makes offline cracking impractical.
- **Password managers** + **never reuse** passwords.

> For most people, the single biggest upgrade is a **password manager + unique passwords + MFA**.

Your turn (finalize)

1. Finalize your **journal of cracked creds (lab)** with commands and labeled screenshots.
2. Complete the **attack → defense** table.
3. Write a **password-defense reflection** (½–1 page): which single defense matters most, and why?

Day 5 exit ticket

Submit the reflection, plus one sentence:

> "The single best password defense for most people is ___ because ___."

🚀 Lab walk-through (Days 2–5)

Platform: Hydra + **John** (+ optional **Hashcat**) on **Kali**, against an authorized **TryHackMe** room or an isolated **lab login service**.

1. See **hashing & salting** in action.
2. **Inspect** `rockyou.txt`.
3. **Hydra** vs. the lab login → recover a credential.
4. **John** vs. sample hashes → watch MD5 fall and bcrypt resist.
5. Build the **attack → defense** table; write the reflection.

> Confirm the lab is **isolated** first. Every Hydra target is a lab/authorized target.

Recap

- Plaintext is a disaster; **hashing** is one-way; **salting** makes identical passwords differ.
- **MD5/SHA** crack fast; **bcrypt** resists by design.
- **Online** (Hydra, noisy) vs. **offline** (John/Hashcat, silent).
- **Wordlists** beat brute force because people are predictable.
- **Spraying** dodges simple lockouts.
- Defenses: **long unique passwords, MFA, lockout/rate-limiting, strong salted hashing.**

Stretch goals

- Use **Hashcat** with a **rule** (`best64.rule`) or a **mask** and explain it.
- Crack a **salted** sample and explain how the tool uses the salt.
- Build a tiny **custom wordlist** and discuss targeted-guessing ethics.
- Estimate crack-time for an 8- vs. 14-character password (show the math).
- Research a real breach where **plaintext/weak hashing** made it worse.

Exit ticket & discussion

Exit ticket: "The single best password defense for most people is ___ because ___."

Discussion: Why did the **bcrypt** sample resist when **MD5** fell fast? And: where exactly is the authorization boundary when someone asks you to crack "their own" account?